HCMUS
Viet Nam National University
Ho Chi Minh City
University of Science

CSC10006 – Introduction to Database

Integrity Constraint & Trigger

Information department – FIT- VNUHCM-US

TABLE OF CONTENTS

1 Objectives

Sau khi hoàn thành bài thực hành này sinh viên sẽ biết được:

- Cài đặt các ràng buộc toàn vẹn trên CSDL sử dụng các kỹ thuật CHECK, RULE, TRIGGER, …

2 Detailed Instructions

2.1 Introduction

Integrity constraints are rules imposed on a database to ensure that the database remains consistent and correct with respect to real-world semantics or human requirements.

Basic components of an integrity constraint:

- Natural language statement of the integrity constraint: a plain-language description of the constraint.
- Context: the relations involved in the integrity constraint. These are considered "relevant" relations because updates to them may cause the constraint to be violated.
- Content: a formal restatement of the integrity constraint to express it more precisely and rigorously. Some formal languages commonly used include relational calculus, relational algebra, or pseudocode.
- Impact table: a table describing how update operations on database tables may affect the integrity constraint.

Types of integrity constraints:

- Domain constraints
- Uniqueness constraints
- Referential constraints
- Inter-attribute constraints within a relation
- Inter-tuple constraints within a relation
- Inter-tuple, inter-relation constraints

- Aggregate attribute constraints

2.2 Basic techniques for implementing integrity constraints

2.2.1 Simple techniques

- NOT NULL
- PRIMARY KEY
- FOREIGN KEY
- CHECK

Use cases:

- **PRIMARY KEY:** Used specifically for primary key constraints. Each table in SQL Server's relational data model can have at most one primary key.
- **UNIQUE:** Used specifically for uniqueness constraints. Attributes declared with a uniqueness constraint can be considered attributes of a candidate key. Each table may define multiple candidate keys.
- **FOREIGN KEY:** Used specifically to create referential integrity constraints or foreign keys. Foreign key attributes must reference key attributes (either a primary key or a candidate key).
- **CHECK:** Used specifically to create other types of integrity constraints. In this case, the constraints are described as conditional expressions that the data must satisfy.

The usage of these techniques is presented in the section on database structure design.

2.2.2 RULE

A RULE is defined as a rule associated with an attribute. The steps for creating and using a RULE are as follows:

Tạo RULE:

```
CREATE RULE [Rule_Name]
AS [Condition_Expression]
```

```
[ ; ]
```

Where:

- [Rule_Name]: the name assigned by the programmer.
- [Condition_Expression]: the expression representing the content of the RULE. In this expression, only one variable (starting with @) may be used to describe the RULE. When the RULE is bound to an attribute, that variable corresponds to the value of that attribute.

Binding a RULE to an attribute:

```
EXEC sp_bindrule [ @rulename = ] 'Rule_Name',
   [ @objname = ] 'Table_Name.Attribute_Name'
```

Note: Students may further explore some other RULE-related syntax, such as using the futureonly parameter or binding a RULE to a data type.

Unbinding a RULE from an attribute:

```
EXEC  sp_unbindrule   [ @objname = ] 'Table_Name.Attribute_Name'
```

Note: When unbinding a RULE from an attribute, all RULEs bound to that attribute are removed.

Delete RULE

```
DROP RULE [Rule_Name]
```

Example 1: Implement the integrity constraint that a teacher's salary must fall within the range **($1000, $20000)** using a **RULE**.

Create a **RULE** to represent a range constraint.

```
CREATE RULE range_rule
AS
@range>= $1000 AND @range <$20000;
```

Bind the created RULE to the Luong attribute of the GIAOVIEN table

```
sp_bindrule 'range_rule' , 'GIAOVIEN.Luong'
```

When this integrity constraint is no longer needed, unbind the **RULE** from the salary attribute.

```
sp_unbindrule 'GIAOVIEN.Luong'
```

2.3 Advanced techniques for implementing integrity constraints: TRIGGER

2.3.1 Introduction

A trigger is a mechanism for enforcing integrity constraints by using the programming capabilities of the Database Management System.

2.3.2 Syntax:

<u>Creating a Trigger</u>

```
CREATE TRIGGER  [Trigger_Name]
ON [Table_Name]
FOR [Operations: INSERT, UPDATE, or DELETE]
AS
IF UPDATE (Attribute_Name)
BEGIN
-- Trigger body: source code for checking or updating
END
```

Important notes on using triggers:

- A trigger is attached to a table to monitor changes to that table's data. The source code in the trigger body is automatically executed whenever the specified data modification operations (INSERT, UPDATE, or DELETE) occur on the table. Therefore, the trigger body usually performs tasks such as checking data, modifying data, or canceling an operation so that integrity constraints are not violated.

- In the trigger body, to make data checking easier, the DBMS provides two temporary tables for the trigger writer to use. These two tables have exactly the same structure as the main table:
    - **inserted:** contains the newly inserted rows
    - **deleted:** contains the rows that have just been deleted
- Note: There is no updated table because an update operation is considered as a combination of a delete and an insert. When an update operation is executed, the inserted table contains the new data, while the deleted table contains the old data.
- In MS SQL Server, a trigger is executed after the corresponding operation (INSERT, UPDATE, DELETE) has been performed on the main table. If the user wants to restore the data in the main table, they can issue the ROLLBACK command. In addition, the RAISERROR function can be used to report an error when an integrity constraint violation is detected. In newer versions of SQL Server, THROW is the newer statement used to raise exceptions, and it is generally recommended over RAISERROR for new development.

Delete Trigger

```
DROP TRIGGER  [Trigger_name]
```

Update Trigger:

```
ALTER TRIGGER  [Trigger_name]]
ON [Table_name]
FOR [Operations: INSERT, UPDATE, or DELETE]
AS
IF UPDATE (Attribute_name)
BEGIN
    ■  -- Trigger body: source code for checking or updating
    -- Case 1: Validation code
    IF (condition under which the integrity constraint is violated )
```

```
    BEGIN
        RAISERROR (N'Error: XXXX', 16, 1)
        ROLLBACK
    END

    -- Case 2: Update code
    UPDATE ...
    ...
END
```

2.3.3 Some examples:

Example 2: Implement the integrity constraint "A teacher's salary must be greater than or equal to 1000" using the trigger technique.

Remark: This integrity constraint is related to the GIAOVIEN table (→ ON GIAOVIEN). When a new row is inserted (INSERT) or when the LUONG attribute is updated (UPDATE), this constraint may be violated. Deleting a row does not affect this constraint (→ FOR INSERT, UPDATE).

	I	D	U
GIAOVIEN	+	-	+ (LUONG)

Use a trigger to validate newly inserted or updated data; if the rule is violated, raise an error and roll back the data.

```
CREATE TRIGGER trgLuong
ON GIAOVIEN
FOR insert, update
AS
if update(LUONG)
BEGIN
```

```
        if exists (select * from inserted where LUONG < 1000)
        begin
                raiserror (N'Lỗi: Lương của giáo viên phải >= 1000 ', 16, 1)
                rollback
        end
END
```

Assume there is a table CTHD(MaHD, MaSP, SoLuong, DonGia, ThanhTien).

Example 3: Implement the following integrity constraint: "ThanhTien must be equal to SoLuong * DonGia."

Remark: This integrity constraint is related to the CTHD table. Whenever a new row is inserted or attributes such as SoLuong or DonGia are updated, the corresponding ThanhTien value must be recalculated accordingly.

	I	D	U
CTHD	+	-	+ (SoLuong, DonGia, ThanhTien)

Use a trigger to update the data so that it complies with the integrity constraint.

```
CREATE TRIGGER trgCapNhatThanhTien
ON CTHD
FOR insert, update
AS
IF update(SoLuong, DonGia, ThanhTien)
BEGIN
        Update CTHD
        Set ThanhTien = SoLuong * DonGia
        Where EXISTS (select * from inserted I where i.MAHD=CTHD.MAHD AND i.MASP =
CTHD.MASP)
END
```

Example 4: Implement the following integrity constraint: "A teacher assigned as the head of a department must be a member of that department."

	I	D	U
GIAOVIEN	-	-	+ (MABM)
BOMON	-	-	+ (TRBOMON)

Assume that the primary key and foreign key constraints have already been defined.

```
CREATE TRIGGER trgTruongBoMon_BOMON
ON BOMON
FOR update
AS
IF update(TRBOMON)
BEGIN
        IF exists (SELECT *
                FROM INSERTED I
                WHERE I.TRBOMON IS NOT NULL
                        AND I.TRBOMON NOT IN  ( SELECT G.MAGV
                                                FROM GIAOVIEN G
                                                WHERE G.MABM=I.MABM ) )
        BEGIN
                raiserror (N'Lỗi: Trưởng bộ môn phải là người trong bộ môn ', 16, 1)
                rollback
        END
END
```

GIAOVIEN

MAGV	...	...	MABM
001			HTTT
002			CNPM
003			HTTT

BOMON

MABM	...	...	TRBOMON
HTTT			002
CNPM			002

GIAOVIEN

MAGV	...	...	MABM
001			HTTT
002			CNPM→ HTTT
003			HTTT

BOMON

MABM	...	...	TRBOMON
HTTT			001
CNPM			002

```
CREATE TRIGGER trgTruongBoMon_GIAOVIEN
ON GIAOVIEN
FOR update
AS
IF update(MABM)
BEGIN
        IF exists (SELECT *
                FROM BOMON B, INSERTED I
                WHERE B.TRBOMON=I.MAGV AND B.MABM <> I.MABM)
        BEGIN
                raiserror (N'Lỗi: Trưởng bộ môn phải là người trong bộ môn ', 16, 1)
                rollback
        END
END
```

3 Exercise

Requirements:

1. Implement some domain integrity constraints using the CHECK and RULE techniques:

   IC 1. A teacher's gender must be either 'Nam' or 'Nữ'.

   IC 2. A teacher's salary must be a multiple of ten.

   IC 3. A teacher's age must be between 18 and 60.

2. Implement the following integrity constraints using TRIGGER

IC 1. The project title must be unique. (Tên đề tài phải duy nhất).

IC 2. The head of department must have been born before 1975.

IC 3. Each department must have at least one female teacher.

IC 4. Each teacher must have at least one phone number.

IC 5. Each teacher may have at most three phone numbers.

IC 6. Each department must have at least four teachers.

IC 7. The head of department must be the oldest person in the department.

IC 8. If a teacher is already the head of a department, that teacher may not serve as another teacher's academic supervisor.

IC 9. A teacher and that teacher's academic supervisor must belong to the same department.

IC 10. Each teacher may have at most one spouse.

IC 11. If a teacher is male, the spouse must be female, and vice versa.

IC 12. If a relative has the relationship "daughter" or "son" to a teacher, then the teacher's year of birth must be earlier than the relative's year of birth.

IC 13. A teacher may supervise at most three projects as the principal investigator.

IC 14. Each project must have at least one task.

IC 15. A teacher's salary must be lower than that of the teacher's supervisor.

IC 16. The salary of the head of department must be higher than the salaries of all teachers in that department.

IC 17. Every department must have a head of department, and the head of department must be a teacher of the institution.

IC 18. A teacher may supervise at most three other teachers.

IC 19. A teacher may only participate in projects whose principal investigator belongs to the same department as that teacher.